

基于 LangChain / LangGraph 官方组件实现智能体上下文

create_agent + 官方摘要中间件 → Redis 热上下文 → Redis Stream → MongoDB 永久归档

基于当前 agent_service 代码编写 · 2026-08-29 · 只提供操作文件，不修改项目源码

最终结论：让官方组件负责上下文，让 Redis 负责主写热路径

本指南删除手写 ContextManager、ContextSnapshot、手工 Token 估算和摘要 Gateway。每轮由 LangGraph checkpoint 保存最新消息状态；Redis 写入成功后才 XADD 归档事件，MongoDB 消费者异步保存终态轮次与 latest 快照。正常热请求不查询 MongoDB。

先看最终运行流程

同一会话 Redis Lock

- Redis checkpoint 命中？
 - 是：只提交本轮 HumanMessage
 - 否：MongoDBStore 读取 latest，恢复历史消息后再提交本轮 HumanMessage
- create_agent + dynamic_prompt + SummarizationMiddleware
- AsyncShallowRedisSaver (durability='sync') 完成最终 checkpoint
- XADD xc:agent:context:persist:v1
- 允许 SSE done / RUN_CANCELLED error

后台单消费者：Pending 优先 → 新消息 → MongoDBStore.aput(turn:{request_id})
→ MongoDBStore.aput(latest) → XACK

层	官方组件 / 基础设施	唯一职责
Agent	LangChain create_agent	标准 messages 状态、模型与工具循环

层	官方组件 / 基础设施	唯一职责
压缩	SummarizationMiddleware	达到 8000 tokens 后摘要，保留最近 12 条消息
热上下文	AsyncShallowRedisSaver	按 thread_id 保存最新图状态，TTL 7 天
异步交接	Redis Stream	在 Redis checkpoint 后记录终态归档事件
永久上下文	MongoDBStore	保存 turn:{request_id} 与 latest

第 0 步：确认上一版哪些内容必须停用

当前仓库可保留 HTTP 路径、Java DTO、SSE 信封、取消接口和角色文件热加载；只替换上下文链路。新旧两套上下文逻辑不能同时工作。

现有内容	问题	本次处理
schemas/context.py	ContextSnapshot / ContextEntry 重复定义消息状态	停用；消息直接使用 LangChain BaseMessage
services/context_manager.py	手写装配、估算、裁剪和摘要	停用；改用 SummarizationMiddleware
services/context_repositor y.py	MongoDB-first, Redis 只是回填缓存	停用；改为 checkpoint → Stream → MongoDB
models/gateway.py complete()	为摘要额外扩展自研接口	停用摘要接口；摘要模型交给官方中间件
手工 revision	与 LangGraph checkpoint_id 重复	停用；热状态由 checkpointer 管理
graph/workflow.py 上下文节点	重复实现 load/compact/persist	改用 create_agent 标准图

边界不要扩大

不复制 LangGraph 的每个内部 checkpoint 到 MongoDB；不引入 Lua、分片、DLQ、向量长期记忆、自研双层 Checkpointer 或第二套摘要器。第一版只运行一个 API worker 和一个归档消费者。

第 1 步：部署 Python Agent 专用 Redis 8

Java 现有 Redis 6.2.6 保持原地址、端口和 DB 不变。Python Agent 另起 Redis 8（或 Redis Stack），固定使用 DB 0，因为 Redis Checkpointer 依赖 RedisJSON 与 RediSearch。

新增运维文件 `deploy/agent-context-redis/compose.yaml`

```
services:
  agent-context-redis:
    image: redis:8.0
    container_name: xc-agent-context-redis
    restart: unless-stopped
    command:
      - redis-server
      - --appendonly
      - "yes"
      - --appendfsync
      - everysec
      - --requirepass
      - ${AGENT_CONTEXT_REDIS_PASSWORD}
    ports:
      # 示例使用宿主机 6380，避免碰到 Java 的 6379。
      - "6380:6379"
    volumes:
      - agent_context_redis_data:/data
    healthcheck:
      test: ["CMD-SHELL", "redis-cli -a $$AGENT_CONTEXT_REDIS_PASSWORD ping | grep PONG"]
      interval: 5s
      timeout: 3s
      retries: 20
    environment:
      AGENT_CONTEXT_REDIS_PASSWORD: ${AGENT_CONTEXT_REDIS_PASSWORD}

volumes:
  agent_context_redis_data:
```

执行命令 `deploy/agent-context-redis/`

```
# PowerShell: 先给当前终端设置密码，再启动专用实例。
$env:AGENT_CONTEXT_REDIS_PASSWORD = "请替换为强密码"
docker compose up -d

# 1. 必须返回 PONG。
redis-cli -h 127.0.0.1 -p 6380 -a "请替换为强密码" -n 0 PING

# 2. Redis 8 应能看到 search/json 相关模块。
redis-cli -h 127.0.0.1 -p 6380 -a "请替换为强密码" -n 0 MODULE LIST

# 3. 初次 setup 前可能为空；执行 checkpointer asetup() 后应出现索引。
redis-cli -h 127.0.0.1 -p 6380 -a "请替换为强密码" -n 0 FT._LIST
```

预期输出

PING 为 PONG；MODULE LIST 中存在 RedisJSON 与 RedisSearch 能力；执行后文 smoke_check.py 后，FT._LIST 至少出现 checkpoint 与 writes 索引。若 MODULE LIST 无相关模块，不要继续启动 Agent。

常见错误

ERR unknown command 'JSON.SET' 或 'FT.CREATE' 表示连到了普通 Redis 6/7，或误用了 Java 的旧实例。检查端口、URL 最后的 /0，以及容器镜像。

第 2 步：增加依赖与环境变量

修改文件 agent_service/pyproject.toml

```
# 在现有 dependencies 中追加；保留项目已有 langchain/langgraph/redis/pymongo。
"langgraph-checkpoint-redis">=0.5.2,<0.6",
"langgraph-store-mongodb">=0.3,<0.4",

# 在 agent_service 根目录执行。
uv lock
uv sync --dev
uv run python -c "from langgraph.checkpoint.redis.ashallow import AsyncShallowRedisSaver; from langgraph.store.mongodb import MongoDBStore; print('imports ok')"
```

追加配置 agent_service/.env.example 与本地 .env

```
# Python Agent 专用 Redis 8，固定 DB 0；不要填 Java Redis 6.2.6 的 URL。
AGENT_CONTEXT_REDIS_URL=redis://:请替换密码@127.0.0.1:6380/0
AGENT_CONTEXT_CHECKPOINT_TTL_MINUTES=10080
AGENT_CONTEXT_CHECKPOINT_REFRESH_ON_READ=true
AGENT_CONTEXT_STREAM_KEY=xc:agent:context:persist:v1
AGENT_CONTEXT_STREAM_GROUP=xc-agent-context-mongodb-v1
AGENT_CONTEXT_STREAM_CONSUMER=worker-1
AGENT_CONTEXT_LOCK_TTL_SECONDS=600
AGENT_CONTEXT_LOCK_WAIT_SECONDS=620

# MongoDB 只做永久归档。MongoDB 短暂不可用不阻断已进入 Stream 的请求。
AGENT_MONGODB_URI=mongodb://mongo:mongo@127.0.0.1:27017/?authSource=admin
AGENT_MONGODB_DATABASE=xinchuang_agent_context
AGENT_MONGODB_CONTEXT_COLLECTION=langgraph_context_archive

# 官方摘要中间件参数。
```

```
AGENT_CONTEXT_SUMMARY_TRIGGER_TOKENS=8000
AGENT_CONTEXT_SUMMARY_KEEP_MESSAGES=12
AGENT_CONTEXT_SUMMARY_INPUT_TOKENS=12000
AGENT_CONTEXT_SUMMARY_MAX_OUTPUT_TOKENS=1200
```

TTL 单位必须写对

langgraph-checkpoint-redis 的 default_ttl 单位是分钟，不是秒。10080 分钟等于 7 天；

refresh_on_read=true 表示热会话每次读取都会续期。

第 3 步：完整替换 Settings，修复类定义阶段引用 self

当前 config.py 把依赖 self 的校验写在类定义体中，导入模块时就会触发 NameError。应把所有跨字段校验放进 model_validator。

替换文件内容 agent_service/src/agent_service/config.py

```
"""应用配置：所有上下文参数集中定义并在实例创建后校验。"""

from functools import lru_cache
from typing import Self

from pydantic import Field, SecretStr, model_validator
from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    """从 AGENT_ 前缀环境变量读取配置。"""

    model_config = SettingsConfigDict(
        env_prefix="AGENT_",
        env_file=".env",
        env_file_encoding="utf-8",
        extra="ignore",
    )

    environment: str = "development"
    log_level: str = "INFO"
    internal_token: SecretStr
    role_config_path: str = "config/roles.yaml"

    # 业务回答模型。继续使用项目现有 OpenAI-compatible 配置。
    model_name: str
    model_api_key: SecretStr
    model_base_url: str | None = None
    model_temperature: float = Field(default=0.2, ge=0, le=2)

    # 摘要模型可与业务模型相同；单独实例化以固定最大输出 1200 tokens。
```

```

context_summary_model_name: str | None = None
context_summary_max_output_tokens: int = Field(default=1200, ge=128, le=4096)

# Python Agent 专用 Redis 8 / Redis Stack, URL 必须以 /0 结束。
context_redis_url: SecretStr
context_checkpoint_ttl_minutes: int = Field(default=10080, ge=1)
context_checkpoint_refresh_on_read: bool = True
context_stream_key: str = "xc:agent:context:persist:v1"
context_stream_group: str = "xc-agent-context-mongodb-v1"
context_stream_consumer: str = "worker-1"
context_lock_ttl_seconds: int = Field(default=600, ge=30)
context_lock_wait_seconds: int = Field(default=620, ge=30)

# MongoDB 是永久归档, 不参与正常热请求。
mongodb_uri: SecretStr
mongodb_database: str = "xinchuang_agent_context"
mongodb_context_collection: str = "langgraph_context_archive"

# 官方 SummarizationMiddleware 参数。
context_summary_trigger_tokens: int = Field(default=8000, ge=1000)
context_summary_keep_messages: int = Field(default=12, ge=2)
context_summary_input_tokens: int = Field(default=12000, ge=1000)

@model_validator(mode="after")
def validate_context_settings(self) -> Self:
    """实例字段填充完成后再做跨字段校验; 类定义阶段绝不引用 self。"""

    redis_url = self.context_redis_url.get_secret_value()
    if not redis_url.rstrip("/").endswith("/0"):
        raise ValueError("AGENT_CONTEXT_REDIS_URL 必须连接专用 Redis 的 DB 0")
    if self.context_summary_input_tokens < self.context_summary_trigger_tokens:
        raise ValueError("摘要输入上限不得小于摘要触发阈值")
    if self.context_lock_wait_seconds < self.context_lock_ttl_seconds:
        raise ValueError("Lock 等待时间应不小于 Lock TTL")
    return self

@lru_cache(maxsize=1)
def get_settings() -> Settings:
    """进程内复用不可变配置; 测试可调用 cache_clear() 后重建。"""

    return Settings()

```

预期输出

```

uv run python -c "from agent_service.config import get_settings;
print(get_settings().context_checkpoint_ttl_minutes)" 输出 10080, 且导入 config.py 不再出现
NameError。

```

第 4 步：用 create_agent、动态角色和官方摘要中间件建图

角色文件仍由现有 RoleProfileProvider 在每次请求开始时读取。system_prompt 通过 runtime context 动态注入，不进入持久 messages，因此角色文件改动不会污染历史。

新增文件 agent_service/src/agent_service/graph/agent_factory.py

```
"""使用 LangChain 官方 Agent 工厂创建标准消息图。"""

from collections.abc import Awaitable, Callable
from dataclasses import dataclass
from typing import Any

from langchain.agents import create_agent
from langchain.agents.middleware import (
    AgentMiddleware,
    ModelRequest,
    ModelResponse,
    SummarizationMiddleware,
    dynamic_prompt,
)
from langchain_openai import ChatOpenAI
from langgraph.checkpoint.base import BaseCheckpointSaver

from agent_service.config import Settings

@dataclass(frozen=True)
class AgentRunContext:
    """只在本次运行可见的数据；不会作为聊天消息写入 checkpoint。"""

    user_id: int
    session_id: int
    request_id: int
    system_prompt: str
    max_output_tokens: int

@dynamic_prompt
def active_role_prompt(request: ModelRequest[AgentRunContext]) -> str:
    """每次模型调用都使用本轮刚加载的活动角色提示词。"""

    return request.runtime.context.system_prompt

class MaxOutputTokensMiddleware(AgentMiddleware):
    """通过官方 ModelRequest 扩展点保留 Java DTO 的 maxOutputTokens。"""

    async def awrap_model_call(
        self,
        request: ModelRequest[AgentRunContext],
        handler: Callable[[ModelRequest[AgentRunContext]], Awaitable[ModelResponse]],
    ) -> ModelResponse:
        settings: dict[str, Any] = dict(request.model_settings)
        settings["max_tokens"] = request.runtime.context.max_output_tokens
        return await handler(request.override(model_settings=settings))

def _chat_model(settings: Settings, *, max_tokens: int | None = None) -> ChatOpenAI:
```

```

"""使用 LangChain 官方 ChatOpenAI, 而不是自研 ModelGateway."""

return ChatOpenAI(
    model=settings.model_name,
    api_key=settings.model_api_key,
    base_url=settings.model_base_url,
    temperature=settings.model_temperature,
    max_tokens=max_tokens,
    streaming=True,
)

def build_agent(settings: Settings, checkpointer: BaseCheckpointSaver):
    """创建一次并在进程内复用的 CompiledStateGraph."""

    answer_model = _chat_model(settings)
    summary_model = ChatOpenAI(
        model=settings.context_summary_model_name or settings.model_name,
        api_key=settings.model_api_key,
        base_url=settings.model_base_url,
        temperature=0,
        max_tokens=settings.context_summary_max_output_tokens,
        streaming=False,
    )

    return create_agent(
        model=answer_model,
        tools=[],
        context_schema=AgentRunContext,
        checkpointer=checkerpoint,
        middleware=[
            active_role_prompt,
            MaxOutputTokensMiddleware(),
            SummarizationMiddleware(
                model=summary_model,
                trigger=("tokens", settings.context_summary_trigger_tokens),
                keep=("messages", settings.context_summary_keep_messages),
                trim_tokens_to_summarize=settings.context_summary_input_tokens,
            ),
        ],
        name="xinchuang-agent",
    )

```

为什么不再写 ContextManager

create_agent 的标准 AgentState 已有 messages reducer; SummarizationMiddleware 会计数、触发、裁剪摘要输入并替换旧历史。其 transformer 会标记并过滤内部摘要模型调用, 因此摘要 token 不应进入用户 SSE。

摘要参数的精确定义

trigger=('tokens', 8000) 表示达到阈值后触发; keep=('messages', 12) 表示摘要后保留最近 12 条消

息；trim_tokens_to_summarize=12000 是送入摘要模型前的输入裁剪上限；summary_model 的 max_tokens=1200 控制摘要最大输出。

第 5 步：初始化 AsyncShallowRedisSaver 热上下文

Shallow Saver 只保留每个 thread 的最新 checkpoint，正好匹配“Redis 快读 + Mongo 永久终态”的目标。thread_id 固定为 user_id:session_id。

新增自检脚本 agent_service/scripts/smoke_check_context_redis.py

```
"""连接专用 Redis 并创建官方 Checkpointer 索引。"""

import asyncio

from langgraph.checkpoint.redis.ashallow import AsyncShallowRedisSaver
from agent_service.config import get_settings

async def main() -> None:
    settings = get_settings()
    ttl = {
        "default_ttl": settings.context_checkpoint_ttl_minutes, # 单位：分钟
        "refresh_on_read": settings.context_checkpoint_refresh_on_read,
    }
    async with AsyncShallowRedisSaver.from_conn_string(
        settings.context_redis_url.get_secret_value(),
        ttl=ttl,
    ) as saver:
        # 上下文管理器已调用 asetup；再次调用是幂等验证，也明确暴露失败位置。
        await saver.asetup()
        print("AsyncShallowRedisSaver asetup ok")

if __name__ == "__main__":
    asyncio.run(main())
```

执行命令 agent_service/

```
uv run python scripts/smoke_check_context_redis.py
redis-cli -h 127.0.0.1 -p 6380 -a "请替换密码" -n 0 FT._LIST
```

失败策略

Redis 不可用时拒绝运行，不允许直接改走 MongoDB 主写。否则同一会话可能同时存在 Redis 新状态与 MongoDB 旧状态，破坏明确的热上下文主路径。

第 6 步：定义终态归档事件和官方消息序列化

新增文件 agent_service/src/agent_service/schemas/context_persistence.py

```
"""Redis Stream 中的终态上下文事件。"""

from datetime import UTC, datetime
from typing import Any, Literal
from uuid import uuid4

from langchain_core.messages import BaseMessage, messages_to_dict
from pydantic import BaseModel, ConfigDict, Field

class ContextPersistEvent(BaseModel):
    """只描述正常或取消终态；不复制每个内部 checkpoint。"""

    model_config = ConfigDict(extra="forbid")

    schema_version: Literal[1] = 1
    event_id: str = Field(default_factory=lambda: str(uuid4()))
    request_id: int
    user_id: int
    session_id: int
    thread_id: str
    context_status: Literal["completed", "cancelled"]
    checkpoint_id: str
    messages: list[dict[str, Any]]
    created_at: datetime = Field(default_factory=lambda: datetime.now(UTC))

    @classmethod
    def from_messages(
        cls,
        *,
        request_id: int,
        user_id: int,
        session_id: int,
        status: Literal["completed", "cancelled"],
        checkpoint_id: str,
        messages: list[BaseMessage],
    ) -> "ContextPersistEvent":
        return cls(
            request_id=request_id,
            user_id=user_id,
            session_id=session_id,
            thread_id=f"{user_id}:{session_id}",
            context_status=status,
            checkpoint_id=checkpoint_id,
            messages=messages_to_dict(messages),
        )
```

不要自己设计 role/content JSON

messages_to_dict() 与 messages_from_dict() 能保存 AIMessage、HumanMessage 及其 metadata；取消轮次的 context_status 与 request_id 因而能够原样恢复。

第 7 步：实现 MongoDBStore 归档服务与 Redis 冷恢复

新增文件 agent_service/src/agent_service/services/context_archive.py

```

"""MongoDB 永久归档与 Redis 未命中时的冷恢复。"""

from typing import Any

from langchain_core.messages import BaseMessage, HumanMessage, messages_from_dict
from langgraph.checkpoint.base import BaseCheckpointSaver
from langgraph.store.base import BaseStore

def graph_config(user_id: int, session_id: int) -> dict[str, Any]:
    """LangGraph Checkpointer 的唯一会话定位配置。"""

    return {"configurable": {"thread_id": f"{user_id}:{session_id}"}}

def archive_namespace(user_id: int, session_id: int) -> tuple[str, ...]:
    """MongoDBStore 的层级 namespace。"""

    return ("agent_context", str(user_id), str(session_id))

class ContextArchiveService:
    def __init__(self, checkpointer: BaseCheckpointSaver, mongo_store: BaseStore) -> None:
        self._checkpointer = checkpointer
        self._mongo_store = mongo_store

    async def input_for_run(
        self,
        *,
        user_id: int,
        session_id: int,
        user_text: str,
    ) -> dict[str, list[BaseMessage]]:
        """Redis 命中只交本轮消息；未命中才从 MongoDB 恢复 latest。"""

        config = graph_config(user_id, session_id)
        hot = await self._checkpointer.aget_tuple(config)
        if hot is not None:
            return {"messages": [HumanMessage(content=user_text)]}

        latest = await self._mongo_store.aget(
            archive_namespace(user_id, session_id),
            "latest",
        )
        if latest is None:

```

```
return {"messages": [HumanMessage(content=user_text)]}

restored = messages_from_dict(latest.value["messages"])
return {"messages": [*restored, HumanMessage(content=user_text)]}
```

正常热请求为什么不查 MongoDB

先用 `checkpointer.aget_tuple()` 做 Redis 命中判断。命中后只给图本轮 `HumanMessage`，`LangGraph` 自动合并 checkpoint 中的历史；只有 Redis TTL 失效或首次启动才读取 MongoDB latest。

第 8 步：先 XADD，再由单消费者异步写 MongoDB

新增文件 agent_service/src/agent_service/services/context_stream.py

```

"""Redis Stream 生产者与 MongoDBStore 单消费者。"""

import asyncio
import logging
from collections.abc import Callable
from typing import Any

from langchain_core.messages import messages_from_dict
from langgraph.store.base import BaseStore
from redis.asyncio import Redis
from redis.exceptions import ResponseError

from agent_service.schemas.context_persistence import ContextPersistEvent
from agent_service.services.context_archive import archive_namespace

logger = logging.getLogger(__name__)

def _stream_id_value(stream_id: str) -> tuple[int, int]:
    """Redis Stream ID 按 milliseconds-sequence 比较，防止旧 latest 覆盖新快照。"""

    milliseconds, sequence = stream_id.split("-", 1)
    return int(milliseconds), int(sequence)

class ContextStreamPublisher:
    def __init__(self, redis: Redis, stream_key: str) -> None:
        self._redis = redis
        self._stream_key = stream_key

    async def publish(self, event: ContextPersistEvent) -> str:
        """XADD 成功即表示事件已由 Redis 接收；此后才允许终态 SSE。"""

        stream_id = await self._redis.xadd(
            self._stream_key,
            {"payload": event.model_dump_json()},
        )
        return stream_id.decode() if isinstance(stream_id, bytes) else stream_id

class ContextArchiveConsumer:
    def __init__(
        self,
        *,
        redis: Redis,
        mongo_store: BaseStore,
        stream_key: str,
        group: str,
        consumer: str,
    ) -> None:
        self._redis = redis
        self._store = mongo_store
        self._stream_key = stream_key
        self._group = group
        self._consumer = consumer

    async def ensure_group(self) -> None:
        """首次启动创建 group；BUSYGROUP 表示已经存在，可忽略。"""

```

```

try:
    await self._redis.xgroup_create(
        self._stream_key,
        self._group,
        id="0-0",
        mkstream=True,
    )
except ResponseError as exc:
    if "BUSYGROUP" not in str(exc):
        raise

async def _archive(self, stream_id: str, event: ContextPersistEvent) -> None:
    namespace = archive_namespace(event.user_id, event.session_id)
    restored = messages_from_dict(event.messages)
    # 最后两个消息就是本轮 HumanMessage 与 AIMessage; 摘要发生时仍以终态顺序为准。
    turn_messages = event.messages[-2:]
    turn_value: dict[str, Any] = {
        "schema_version": event.schema_version,
        "event_id": event.event_id,
        "stream_id": stream_id,
        "request_id": event.request_id,
        "context_status": event.context_status,
        "checkpoint_id": event.checkpoint_id,
        "messages": turn_messages,
        "created_at": event.created_at.isoformat(),
    }
    await self._store.aput(
        namespace,
        f"turn:{event.request_id}",
        turn_value,
        index=False,
    )

    latest = await self._store.aget(namespace, "latest")
    latest_id = None if latest is None else latest.value.get("stream_id")
    if latest_id is None or _stream_id_value(stream_id) >= _stream_id_value(latest_id):
        await self._store.aput(
            namespace,
            "latest",
            {
                "schema_version": event.schema_version,
                "stream_id": stream_id,
                "request_id": event.request_id,
                "context_status": event.context_status,
                "checkpoint_id": event.checkpoint_id,
                "messages": event.messages,
                "message_count": len(restored),
                "created_at": event.created_at.isoformat(),
            },
            index=False,
        )

async def _consume_batch(self, start_id: str, block_ms: int | None) -> int:
    """start_id='0' 读取本消费者 Pending; '>' 读取从未投递的新消息。"""

    streams = await self._redis.xreadgroup(
        groupname=self._group,
        consumername=self._consumer,
        streams={self._stream_key: start_id},
        count=20,
        block=block_ms,
    )
    handled = 0
    for _, records in streams:
        for raw_id, fields in records:
            stream_id = raw_id.decode() if isinstance(raw_id, bytes) else raw_id
            raw_payload = fields.get(b"payload", fields.get("payload"))

```

```

        try:
            event = ContextPersistEvent.model_validate_json(raw_payload)
            await self._archive(stream_id, event)
            # 两次 MongoDB aput 都成功后才确认。
            await self._redis.xack(self._stream_key, self._group, stream_id)
            handled += 1
        except Exception:
            # 不 XACK; 保留 Pending, 等待 MongoDB 恢复或人工修复事件格式。
            logger.exception("上下文归档失败, stream_id=%s", stream_id)
        return handled

    async def run_forever(self) -> None:
        await self.ensure_group()
        while True:
            # 第一版只有一个固定 consumer 名称; 重启先把自己的 Pending 顺序重放完。
            while await self._consume_batch("", block_ms=None):
                pass
            await self._consume_batch(">", block_ms=5000)
            await asyncio.sleep(0)

```

至少一次 + 幂等

MongoDB 写成功前不 XACK; 进程崩溃后相同 Stream 记录会再次投递。turn:{request_id} 是固定 key, 重复 aput 只覆盖同一轮; latest 还比较 Stream ID, 旧记录不会覆盖新快照。

第一版的有意限制

固定一个 API worker、一个 consumer 名称 worker-1。同一 Stream 按顺序处理, 不做 XAUTOCLAIM、分片、Lua、DLQ 或复杂消费者治理; 扩容前必须重新设计按会话分片和 Pending 转移。

第 9 步: 实现同会话串行、官方 messages 流和正常收尾

替换文件内容 agent_service/src/agent_service/services/agent_runtime.py

```

"""运行协调器: Redis 热状态是唯一同步主路径。"""

from collections.abc import AsyncIterator
from typing import Any

from langchain_core.messages import AIMessage, AIMessageChunk
from redis.asyncio import Redis

from agent_service.core.cancellation import CancellationRegistry
from agent_service.core.exceptions import AgentCancelledError, EmptyModelOutputError

```

```

from agent_service.graph.agent_factory import AgentRunContext
from agent_service.schemas.chat import AgentEvent, ChatStreamRequest
from agent_service.schemas.context_persistence import ContextPersistEvent
from agent_service.schemas.role import RoleProfile
from agent_service.services.context_archive import ContextArchiveService, graph_config
from agent_service.services.context_stream import ContextStreamPublisher
from agent_service.services.role_profile import RoleProfileProvider

```

```

CANCELLED_NOTICE = "【系统说明：用户已取消本轮回答，以上内容可能不完整或不正确。】"

```

```

def _chunk_text(content: Any) -> str:
    """兼容字符串和标准内容块；只提取可展示文本。"""

    if isinstance(content, str):
        return content
    if isinstance(content, list):
        return "".join(
            block.get("text", "")
            for block in content
            if isinstance(block, dict) and block.get("type") == "text"
        )
    return ""

```

```

class AgentRuntime:
    def __init__(
        self,
        *,
        agent,
        redis: Redis,
        archive: ContextArchiveService,
        publisher: ContextStreamPublisher,
        role_profile_provider: RoleProfileProvider,
        cancellation_registry: CancellationRegistry,
        model_name: str,
        lock_ttl_seconds: int,
        lock_wait_seconds: int,
    ) -> None:
        self.agent = agent
        self.redis = redis
        self.archive = archive
        self.publisher = publisher
        self.role_profile_provider = role_profile_provider
        self.cancellation_registry = cancellation_registry
        self.model_name = model_name
        self.lock_ttl_seconds = lock_ttl_seconds
        self.lock_wait_seconds = lock_wait_seconds

    def load_active_role(self) -> RoleProfile:
        """保留角色文件热加载：每次 HTTP 请求建立 SSE 前调用。"""

        return self.role_profile_provider.load()

    async def _persist_event(
        self,
        *,
        request: ChatStreamRequest,
        status: str,
        config: dict[str, Any],
    ) -> None:
        """读取刚同步落入 Redis 的最终 state，再 XADD 归档事件。"""

        snapshot = await self.agent.aget_state(config)
        checkpoint_id = snapshot.config["configurable"]["checkpoint_id"]
        event = ContextPersistEvent.from_messages(

```

```

        request_id=request.request_id,
        user_id=request.user_id,
        session_id=request.session_id,
        status=status,
        checkpoint_id=checkpoint_id,
        messages=list(snapshot.values["messages"]),
    )
    await self.publisher.publish(event)

async def stream(
    self,
    request: ChatStreamRequest,
    role_profile: RoleProfile,
) -> AsyncIterator[AgentEvent]:
    thread_id = f"{request.user_id}:{request.session_id}"
    lock = self.redis.lock(
        f"xc:agent:context:lock:{thread_id}",
        timeout=self.lock_ttl_seconds,
        blocking_timeout=self.lock_wait_seconds,
    )
    acquired = await lock.acquire()
    if not acquired:
        raise TimeoutError("等待同会话上一轮收尾超时")

    await self.cancellation_registry.register(request.request_id)
    config = graph_config(request.user_id, request.session_id)
    run_context = AgentRunContext(
        user_id=request.user_id,
        session_id=request.session_id,
        request_id=request.request_id,
        system_prompt=role_profile.system_prompt,
        max_output_tokens=request.policy.max_output_tokens,
    )
    answer_parts: list[str] = []
    cancelled = False
    try:
        graph_input = await self.archive.input_for_run(
            user_id=request.user_id,
            session_id=request.session_id,
            user_text=request.message,
        )
        yield AgentEvent(event="status", payload={"stage": "generation", "message": "正在生成回答"})

        async for part in self.agent.astream(
            graph_input,
            config=config,
            context=run_context,
            stream_mode="messages",
            version="v2",
            durability="sync",
        ):
            if await self.cancellation_registry.is_cancelled(request.request_id):
                cancelled = True
                break
            if part["type"] != "messages":
                continue
            chunk, metadata = part["data"]
            # create_agent 的用户回答节点名为 model; 摘要内部调用还会被官方 transformer 过滤。
            if metadata.get("langgraph_node") != "model" or not isinstance(chunk, AIMessageChunk):
                continue
            text = _chunk_text(chunk.content)
            if text:
                answer_parts.append(text)
                yield AgentEvent(event="delta", payload={"content": text})

    if cancelled:

```

```

        partial = "".join(answer_parts)
        content = f"{partial}"

{CANCELLED_NOTICE}" if partial else CANCELLED_NOTICE
        cancelled_message = AIMessage(
            content=content,
            additional_kwargs={
                "context_status": "cancelled",
                "request_id": request.request_id,
            },
        )
        # input checkpoint 已因 durability='sync' 落地; 这里追加取消终态 AIMessage。
        await self.agent.aupdate_state(
            config,
            {"messages": [cancelled_message]},
            as_node="model",
        )
        await self._persist_event(request=request, status="cancelled", config=config)
        raise AgentCancelledError(f"requestId={request.request_id} 已取消")

    if not "".join(answer_parts).strip():
        raise EmptyModelOutputError("模型未生成有效回答")

    # astream 正常结束且 durability='sync': 最终 Redis checkpoint 已完成。
    await self._persist_event(request=request, status="completed", config=config)
    finally:
        await self.cancellation_registry.finish(request.request_id)
        if await lock.owned():
            await lock.release()

    async def cancel(self, request_id: int) -> bool:
        return await self.cancellation_registry.cancel(request_id)

```

严格时序

正常：最终 Redis checkpoint → XADD → runtime.stream 结束 → 路由发送 done。取消：

aupdate_state 追加部分回答和固定说明 → XADD → 抛 AgentCancelledError → 路由发送现有

RUN_CANCELLED error。下一请求只有在 finally 释放会话 Lock 后才能进入。

终态 SSE 的含义

done 与 RUN_CANCELLED error 只表示 Redis checkpoint 已同步落地且 Redis Stream 已接收归档事件；不表示 MongoDB 已完成写入。其他 MODEL_UNAVAILABLE / OUTPUT_VALIDATION_FAILED error 仍表示运行失败，不产生正常或取消终态归档。

第 10 步：保持现有路由契约，只改模型名读取

HTTP 路径仍是 `/internal/ai/v1/chat/stream` 与 `/internal/ai/v1/chat/requests/{requestId}/cancel`；Java DTO、`meta/status/delta/done/error` 事件名都不变。

局部修改 `agent_service/src/agent_service/api/routes/chat.py`

```
# 其余路由逻辑保持不变：仍在建立 SSE 前调用 load_active_role()。
# 正常 done 的 model 字段只把旧网关属性改为运行时属性。
yield frame(
    "done",
    {
        "finishReason": "stop",
        "model": runtime.model_name,
        "roleName": active_role.name,
    },
)

# AgentRuntime 只有在 Redis checkpoint 与 XADD 都成功后才正常返回；
# 取消也只有在 aupdate_state 与 XADD 成功后才抛 AgentCancelledError。
except AgentCancelledError:
    yield frame(
        "error",
        {"code": "RUN_CANCELLED", "message": "智能体运行已取消", "retryable": False},
    )
```

第 11 步：用 FastAPI lifespan 统一管理资源与后台消费者

替换 `create_app/lifespan` 相关内容 `agent_service/src/agent_service/main.py`

```
"""FastAPI 应用工厂：连接、官方组件和单消费者都按进程生命周期管理。"""

import asyncio
import logging
from contextlib import AsyncExitStack, asynccontextmanager

from fastapi import FastAPI
from langgraph.checkpoint.redis.ashallow import AsyncShallowRedisSaver
from langgraph.store.mongodb import MongoDBStore
from redis.asyncio import Redis

from agent_service.api.routes import chat, health
from agent_service.config import Settings, get_settings
from agent_service.core.cancellation import CancellationRegistry
from agent_service.graph.agent_factory import build_agent
from agent_service.services.agent_runtime import AgentRuntime
from agent_service.services.context_archive import ContextArchiveService
from agent_service.services.context_stream import ContextArchiveConsumer, ContextStreamPublisher
from agent_service.services.role_profile import RoleProfileProvider

logger = logging.getLogger(__name__)

def create_app(*, settings: Settings | None = None) -> FastAPI:
    resolved = settings or get_settings()

    @asynccontextmanager
    async def lifespan(app: FastAPI):
        async with AsyncExitStack() as stack:
            redis_url = resolved.context_redis_url.get_secret_value()
            redis = Redis.from_url(redis_url, decode_responses=False)
            await redis.ping() # Redis 是主写路径，探测失败就拒绝启动。
            stack.push_async_callback(redis.aclose)

            checkpointer = await stack.enter_async_context(
                AsyncShallowRedisSaver.from_conn_string(
                    redis_url,
                    ttl={
                        "default_ttl": resolved.context_checkpoint_ttl_minutes,
                        "refresh_on_read": resolved.context_checkpoint_refresh_on_read,
                    },
                )
            )
            # from_conn_string.__aenter__ 已执行 asetup(); 这里不再自建索引。

            mongo_store = stack.enter_context(
                MongoDBStore.from_conn_string(
                    conn_string=resolved.mongodb_uri.get_secret_value(),
                    db_name=resolved.mongodb_database,
                    collection_name=resolved.mongodb_context_collection,
                )
            )

            agent = build_agent(resolved, checkpointer)
            archive = ContextArchiveService(checkpointer, mongo_store)
            publisher = ContextStreamPublisher(redis, resolved.context_stream_key)
            consumer = ContextArchiveConsumer(
                redis=redis,
                mongo_store=mongo_store,
```

```

        stream_key=resolved.context_stream_key,
        group=resolved.context_stream_group,
        consumer=resolved.context_stream_consumer,
    )
    cancellation = CancellationRegistry()
    app.state.agent_runtime = AgentRuntime(
        agent=agent,
        redis=redis,
        archive=archive,
        publisher=publisher,
        role_profile_provider=RoleProfileProvider(resolved.role_config_path),
        cancellation_registry=cancellation,
        model_name=resolved.model_name,
        lock_ttl_seconds=resolved.context_lock_ttl_seconds,
        lock_wait_seconds=resolved.context_lock_wait_seconds,
    )

    consumer_task = asyncio.create_task(consumer.run_forever(), name="context-mongodb-archive")
    try:
        yield
    finally:
        consumer_task.cancel()
        await asyncio.gather(consumer_task, return_exceptions=True)

    app = FastAPI(title="XinChuang Agent Service", lifespan=lifespan)
    app.include_router(health.router, prefix="/internal/ai/v1")
    app.include_router(chat.router, prefix="/internal/ai/v1")
    return app

app = create_app()

```

MongoDB 启动策略

MongoDBStore 连接上下文在 lifespan 中创建，但归档失败不应让已经 XADD 的请求回滚；消费者记录异常并保留 Pending。若希望 MongoDB 彻底离线时 API 也能启动，可把 Store 的首次连接放进消费者重试循环；不要因此绕过 Redis。

第 12 步：测试夹具使用 InMemorySaver 与内存归档

修改文件 agent_service/tests/conftest.py

```

"""上下文测试不连接开发者 Redis/MongoDB。"""

import pytest
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.store.memory import InMemoryStore

@pytest.fixture
def memory_checkpointer() -> InMemorySaver:

```

```

    return InMemorySaver()

@pytest.fixture
def memory_archive() -> InMemoryStore:
    return InMemoryStore()

```

新增单元测试 agent_service/tests/test_context_components.py

```

from unittest.mock import AsyncMock

from langchain_core.messages import AIMessage, HumanMessage, messages_from_dict

from agent_service.schemas.context_persistence import ContextPersistEvent
from agent_service.services.context_archive import ContextArchiveService

async def test_hot_checkpoint_does_not_read_mongodb(memory_checkpointer, memory_archive):
    """第二轮 Redis 命中时, 不允许调用 MongoDBStore.aget。"""

    service = ContextArchiveService(memory_checkpointer, memory_archive)
    # 用官方 checkpointer 写入最小 checkpoint 的细节交给 Agent 集成测试;
    # 此测试通过 monkeypatch 让 aget_tuple 明确返回命中对象。
    memory_checkpointer.aget_tuple = AsyncMock(return_value=object())
    memory_archive.aget = AsyncMock(side_effect=AssertionError("热请求不应访问 MongoDB"))

    result = await service.input_for_run(user_id=7, session_id=3, user_text="第二轮")
    assert result == {"messages": [HumanMessage(content="第二轮")]}
    memory_archive.aget.assert_not_awaited()

async def test_cold_restore_uses_official_message_deserializer(memory_checkpointer, memory_archive):
    memory_checkpointer.aget_tuple = AsyncMock(return_value=None)
    event = ContextPersistEvent.from_messages(
        request_id=21,
        user_id=7,
        session_id=3,
        status="completed",
        checkpoint_id="cp-1",
        messages=[HumanMessage("第一轮"), AIMessage("第一轮回答")],
    )
    await memory_archive.aput(
        ("agent_context", "7", "3"),
        "latest",
        {"messages": event.messages},
        index=False,
    )

    service = ContextArchiveService(memory_checkpointer, memory_archive)
    result = await service.input_for_run(user_id=7, session_id=3, user_text="第二轮")
    assert [m.content for m in result["messages"]] == ["第一轮", "第一轮回答", "第二轮"]

def test_cancelled_message_metadata_round_trip():
    event = ContextPersistEvent.from_messages(
        request_id=22,
        user_id=7,
        session_id=3,
        status="cancelled",
        checkpoint_id="cp-2",
    )

```

```

    messages=[
        HumanMessage("请继续"),
        AIMessage(
            "部分回答

【系统说明：用户已取消本轮回答，以上内容可能不完整或不正确。】",
            additional_kwargs={"context_status": "cancelled", "request_id": 22},
        ),
    ],
)
restored = messages_from_dict(event.messages)
assert restored[-1].additional_kwargs["context_status"] == "cancelled"
assert restored[-1].additional_kwargs["request_id"] == 22

```

测试断言重点

用 AsyncMock 明确断言“热请求没有 MongoDB 读取”，不能只断言最终文本正确；否则 MongoDB-first 回归也可能悄悄通过测试。

第 13 步：集成测试与验收顺序

新增集成测试 agent_service/tests/integration/test_context_persistence.py

```

import asyncio

import pytest
from langchain_core.messages import messages_from_dict

@pytest.mark.integration
async def test_two_turns_use_redis_without_mongo_read(running_app, mongo_spy):
    await running_app.chat(user_id=7, session_id=3, request_id=21, message="第一轮")
    mongo_spy.reset_mock()
    await running_app.chat(user_id=7, session_id=3, request_id=22, message="第二轮")
    mongo_spy.aget.assert_not_called()

@pytest.mark.integration
async def test_mongo_outage_keeps_stream_pending(running_app, redis_client, stop_mongo, start_mongo):
    await stop_mongo()
    await running_app.chat(user_id=7, session_id=4, request_id=31, message="写入待归档事件")
    pending = await redis_client.xpending(running_app.stream_key, running_app.group)
    assert pending["pending"] >= 1
    await start_mongo()
    await running_app.wait_until_archived(request_id=31)
    pending = await redis_client.xpending(running_app.stream_key, running_app.group)
    assert pending["pending"] == 0

@pytest.mark.integration
async def test_cancelled_turn_is_visible_to_next_request(running_app):

```

```
run = asyncio.create_task(
    running_app.chat(user_id=7, session_id=5, request_id=41, message="生成较长回答")
)
await running_app.wait_for_first_delta(41)
await running_app.cancel(41)
await run
next_result = await running_app.chat(
    user_id=7, session_id=5, request_id=42, message="说明上一轮为什么不完整"
)
assert "用户已取消本轮回答" in next_result.model_input_text
```

夹具职责

running_app 负责启动真实 Redis 8 与 MongoDB、注入可预测的测试模型并封装 SSE；mongo_spy 只统计 aget 调用。不要让单元测试连接开发数据库。集成测试可以使用 Testcontainers 或独立 docker compose，但端口和数据卷必须与开发环境隔离。

必须逐项通过的验收矩阵

场景	操作	必须观察到
正常两轮	同 userId/sessionId 连续请求	第二轮从 Redis checkpoint 自动取得第一轮；请求路径 Mongo aget=0
Redis TTL 后恢复	删除/过期该 thread checkpoint 后再请求	只在未命中时读取 Mongo latest，本轮结束重新形成热 checkpoint
正常时序	采集 checkpoint、XADD、done 时间	严格 checkpoint < XADD < done
取消时序	首个 delta 后取消并立即发下一请求	部分回答+固定说明写入；下一请求等待 Lock 释放
MongoDB 停机	停库后完成正常请求	请求仍在 XADD 后终态；事件留 Pending，恢复后写 turn/latest 并 XACK

场景	操作	必须观察到
重复投递	同 Stream 记录处理两次	turn key 不重复；旧 Stream ID 不覆盖 latest
自动摘要	构造超过 8000 tokens 的多轮消息	官方中间件触发；保留 12 条；摘要模型 token 不出现在 SSE delta
并发	同会话并发、不同会话并发	同会话严格串行；不同 thread_id 可同时运行

第 14 步：上线前按固定顺序执行

执行命令 agent_service/

```
# 1. 依赖与静态质量。
uv sync --dev
uv run ruff format .
uv run ruff check .
uv run pytest -m "not integration"

# 2. 基础设施自检。
uv run python scripts/smoke_check_context_redis.py
redis-cli -h 127.0.0.1 -p 6380 -a "请替换密码" -n 0 FT._LIST

# 3. 启动单 worker。第一版不要写 --workers 2 或更大。
uv run uvicorn agent_service.main:app --host 0.0.0.0 --port 8001 --workers 1

# 4. 另一终端观察 Stream 与 Pending。
redis-cli -h 127.0.0.1 -p 6380 -a "请替换密码" -n 0 XINFO GROUPS xc:agent:context:persist:v1
redis-cli -h 127.0.0.1 -p 6380 -a "请替换密码" -n 0 XPENDING xc:agent:context:persist:v1 xc-agent-context-mongodb-v1

# 5. 最后执行真实基础设施集成测试。
uv run pytest -m integration
```

- 确认 Java Redis 6.2.6 的地址、端口与配置完全未改。
- 确认 Python AGENT_CONTEXT_REDIS_URL 指向专用 Redis 8 的 DB 0。
- 确认旧 ContextManager、ContextSnapshot、ContextRepository 与 summary complete() 不再被导入。
- 确认只有一个 API worker、一个固定 consumer，并配置持久 AOF/数据卷。
- 确认日志包含 thread_id、request_id、checkpoint_id、stream_id，但绝不输出密码或完整提示词。
- 确认备份/恢复演练以 MongoDB latest 为永久恢复源，而不是复制 Redis 内部 key。

故障排查：先按症状定位

现象	最可能原因	先检查
启动时报 JSON.SET / FT.CREATE 不存在	连接了普通 Redis 或 Java Redis 6.2.6	端口、/0、MODULE LIST
asetup 后 FT._LIST 为空	RedisJSON/RediSearch 不可用或权限不足	Redis 8 镜像、ACL、日志
第二轮像首次会话	thread_id 不稳定或请求未用同一 userId/sessionId	configurable.thread_id 与 Redis checkpoint
正常请求查 MongoDB	先做了冷恢复，或热命中判断错误	aget_tuple 调用顺序与测试 spy
MongoDB 恢复后仍有 Pending	consumer 名称变化、写入继续失败或未先读 0	XINFO CONSUMERS、异常日志
取消后下一轮看不到部分回答	未执行 aupdate_state，或 XADD 前释放 Lock	取消分支严格时序
摘要文本出现在用户 delta	未使用官方中间件，或转发了非 model 节点	middleware 配置和 langgraph_node 过滤
旧快照覆盖 latest	未按 Stream ID 比较或多消费者打乱顺序	latest.stream_id 与单消费者限制
Lock 意外过期导致同会话重叠	模型运行超过 600 秒	提高 TTL，或上线锁续租后再放宽超时

不要用 MongoDB 兜底 Redis 故障

Redis 是本方案的同步主写与同会话串行基础。Redis 故障时继续运行会失去 checkpoint、Lock 和 Stream 三个一致性支点；正确行为是拒绝运行并返回可重试错误。

官方资料与版本依据

- LangGraph Persistence: <https://docs.langchain.com/oss/python/langgraph/persistence>
- LangGraph Add memory: <https://docs.langchain.com/oss/python/langgraph/add-memory>
- LangGraph Streaming: <https://docs.langchain.com/oss/python/langgraph/streaming>
- LangChain create_agent:
https://reference.langchain.com/python/langchain/agents/factory/create_agent
- dynamic_prompt:
https://reference.langchain.com/python/langchain/agents/middleware/types/dynamic_prompt
- SummarizationMiddleware:
<https://reference.langchain.com/python/langchain/agents/middleware/summarization/SummarizationMiddleware>
- Redis Checkpointer: <https://github.com/redis-developer/langgraph-redis>
- MongoDBStore: <https://www.mongodb.com/docs/atlas/ai-integrations/langgraph/>

版本锁定说明

本指南按 langgraph-checkpoint-redis 0.5.x 与 langgraph-store-mongodb 0.3.x 编写。升级到下一个 minor 范围前，先重新核对构造函数、Stream v2 形态和摘要中间件参数，并运行全部集成测试。

完成标准

- 项目上下文只由 create_agent 标准 messages、SummarizationMiddleware 和 AsyncShallowRedisSaver 管理。
- 热请求只访问 Redis；Redis 未命中才用 MongoDB latest 恢复。
- 正常与取消都严格先完成 Redis checkpoint，再 XADD，再产生终态 SSE。
- MongoDB 通过单消费者异步写入 turn:{request_id} 与 latest，成功后才 XACK。
- 取消轮次保存部分回答、固定取消说明和 metadata，下一轮能够看到。
- 旧上下文实现全部停止导入；Java DTO、HTTP 路径、SSE 名称与角色热加载保持不变。